What is __pycache__?

Asked 10 years ago Modified 1 year, 2 months ago Viewed 582k times

1038

From what I understand, a cache is an encrypted file of similar files.

What do we do with the `__pycache__` folder? Is it what we give to people instead of our source code? Is it just my input data? This folder keeps getting created, what it is for?

python python-3.x caching

Share Follow

edited Jul 12, 2018 at 18:52
Guillem Cucurull
1,681 1 22 30

asked Jun 1, 2013 at 4:24
user2063042
10.4k 3 14 6

45 "Is it what we give people instead of our source code?" - No, you give them the source code in a nice installable package so it's easy to use. – Lennart Regebro Jun 1, 2013 at 12:49

157 Nobody mentioned yet, but your definition of cache is bizarre. Cache is quite simply a component that stores data so future requests for that data can be served faster. – Ricardo Magalhães Cruz Jul 22, 2016 at 19:02

10 See: python.org/dev/peps/pep-3147 – Mr_and_Mrs_D Apr 13, 2017 at 18:47

7 Since `Python 3.8` you can use an environment variable to **change the location** for the annoying cache directories: stackoverflow.com/a/57414308/1612318 – Rotareti Aug 8, 2019 at 14:02

5 A cache is something that keeps a copy of stuff in case you need it again, to save you having to go back to the original to get it. It's designed to be faster than going to the original place. It might be faster because it doesn't have to pre-process or compile the information. Or it might be faster storage, e.g. a disk cache in RAM or a web cache on a local disk. It's not by its nature encrypted (although it could sometimes be), and it's not always a "file of similar files" - it can be a file, a load of files, a block of RAM etc. – rjmunro Feb 4, 2020 at 17:39

11 Answers

Sorted by: Highest score (default)

904

When you run a program in Python, the interpreter compiles it to bytecode first (this is an oversimplification) and stores it in the `__pycache__` folder. If you look in there you will find a bunch of files sharing the names of the `.py` files in your project's folder, only their extensions will be either `.pyc` or `.pyo`. These are bytecode-compiled and optimized bytecode-compiled versions of your program's files, respectively.

As a programmer, you can largely just ignore it... All it does is make your program start a little faster. When your scripts change, they will be recompiled, and if you delete the files or the whole folder and run your program again, they will reappear (unless you specifically suppress that behavior).

When you're sending your code to other people, the common practice is to delete that folder, but it doesn't really matter whether you do or don't. When you're using version control (`git`), this folder is typically listed in the ignore file (`.gitignore`) and thus not included.

If you are using CPython (which is the most common, as it's the reference implementation) and you don't want that folder, then you can suppress it by starting the interpreter with the -B flag, for example

```
python -B foo.py
```

Another option, as noted by tcaswell, is to set the environment variable `PYTHONDONTWRITEBYTECODE` to any value (according to Python's man page, any "non-empty string").

Share Follow

edited Jan 11, 2022 at 17:43
bad_coder - on strike
11k 20 42 71

answered Jun 1, 2013 at 4:31
scott_fakename
10.1k 2 15 20

72 You can also add the environment variable `PYTHONDONTWRITEBYTECODE=<any_value>` to suppress it permanently. – Mark Tolonen Jun 1, 2013 at 15:29

18 Just to clarify, this is for Python 3 only, correct? – Joe J Feb 24, 2014 at 22:39

16 @JoeJ yes, I think that's true. `python2` puts the compiled files in the same directory as the originals, if I'm not mistaken. – scott_fakename Feb 25, 2014 at 1:20

44 One IMPORTANT caveat is that the cached .pyc file will be used instead of the .py file if the .py file is missing. In practice, this only happens if you delete (or rename) modules, so it's not a common occurance, but if some stuff keeps up being "there", after scratching your head, running find . -name *.pyc | xargs rm o n your source is probably a good first reaction. – yacc143 Feb 28, 2014 at 10:38

48 `find . -name '*.pyc' -delete` Yes, find has a flag for deleting the found files, so you don't have to use any xargs shananigans – vlad-ardelean Oct 7, 2014 at 18:58

240

`__pycache__` is a folder containing **Python 3 bytecode compiled and ready to be executed**.

I don't recommend routinely laboriously deleting these files or suppressing creation during development as it wastes your time. Just have a recursive command ready (see below) to clean up when needed as bytecode can become stale in edge cases (see comments).

Python programmers usually ignore bytecode. Indeed `__pycache__` and `*.pyc` are common lines to see in `.gitignore` files. Bytecode is not meant for distribution and can be disassembled using

`dis` module.

If you are using OS X you can easily hide all of these folders in your project by running following command from the root folder of your project.

```
find . -name '__pycache__' -exec chflags hidden {} \;
```

Replace `__pycache__` with `*.pyc` for Python 2.

This sets a flag on all those directories (.pyc files) telling Finder/Textmate 2 to exclude them from listings. Importantly the bytecode is there, it's just hidden.

Rerun the command if you create new modules and wish to hide new bytecode or if you delete the hidden bytecode files.

On Windows the equivalent command might be (not tested, batch script welcome):

```
dir * /s/b | findstr __pycache__ | attrib +h +s +r
```

Which is same as going through the project hiding folders using right-click > hide...

Running unit tests is one scenario (more in comments) where deleting the `*.pyc` files and `__pycache__` folders is indeed useful. I use the following lines in my `~/.bash_profile` and just run `cl` to clean up when needed.

```
alias cpy='find . -name "__pycache__" -delete'
alias cpc='find . -name "*.pyc"       -delete'
...
alias cl='cpy && cpc && ...'
```

and more lately

```
# pip install pyclean
pyclean .
```

Share Follow

edited Mar 24, 2021 at 1:12

answered Feb 6, 2015 at 11:52

fmalina
6,073 4 37 47

2 @DoTheEvo: it simply doesn't get created, so there's no speedup next time the module is loaded. No error is raised. – Petr Viktorin Jul 12, 2015 at 13:31

20 This isn't a good answer. The asker wants to know what these files are for. This answer says "don't worry about it" and then makes them disappear. – interestinglythere Oct 6, 2015 at 19:25

59 ***Absolutely*** *bother deleting these*: it is not pointless. Python will happily not detect file changes and run off a cache file in many circumstances, driving you up the wall with "why the f does it still not work, I changed the code, why is it still failing on nonexistent calls" nonsense. Especially in test frameworks, pycache-by-default is the worst. – Mike 'Pomax' Kamermans May 11, 2017 at 23:12

1 What circumstances? Over the past 10 years, everytime I suspected bytecode to be the problem it was actually somewhere else. – fmalina May 12, 2017 at 17:25

3 I would disagree with this advice to "not bother deleting these files" - have seen this recommended many times, most recently by Kenneth Reitz's "How To Python" ("the bytecode trick") – Louis Maddox Jan 20, 2018 at 22:47

81

A `__pycache__` folder is created when you use the line:

```
import file_name
```

or try to get information from another file you have created. This makes it a little faster when running your program the second time to open the other file.

Share Follow

edited Jan 12, 2020 at 11:02
funnydman
8,703 4 37 52

answered Feb 17, 2018 at 20:49
Aliaksandr S.
1,044 9 20

40

Updated answer from 3.7+ docs:

> To speed up loading modules, **Python caches the compiled version of each module in the `__pycache__` directory** under the name `module.version.pyc`, where the version encodes the format of the compiled file; it generally contains the Python version number. For example, in CPython release 3.3 the compiled version of spam.py would be cached as `__pycache__/spam.cpython-33.pyc`. This naming convention allows compiled modules from different releases and different versions of Python to coexist.

Source: https://docs.python.org/3/tutorial/modules.html#compiled-python-files

That is, this directory is generated by Python and exists to make your programs run faster. It shouldn't be committed to source control, and should coexist in peace with your local source code.

`__pycache__` is a directory that contains bytecode cache files that are automatically generated by python, namely compiled python, or `.pyc`, files. You might be wondering why Python, an

"interpreted" language, has any compiled files at all. This SO question addresses that (and it's definitely worth reading this answer).

The python docs go into more depth about exactly how it works and why it exists:

- It was added in python 3.2 because the existing system of maintaining `.pyc` files in the same directory caused various problems, such as when a program was run with Python interpreters of different versions. For the full feature spec, see PEP 3174.

Share Follow

edited Aug 5, 2019 at 23:43

answered Jan 8, 2018 at 2:43

FluxLemur
1,807 16 18

This is the main part I needed to know: **"It shouldn't be committed to source control"**. Thanks! I'm adding `__pycache__/` to my repos' `.gitignore` file at the repo root now to cause Git to ignore all of these auto-generated `__pycache__` directories! – Gabriel Staples May 8 at 3:39

11

When you **import a module**,

```
import file_name
```

Python stores the compiled bytecode in `__pycache__` directory so that future imports can use it directly, rather than having to parse and compile the source again.

It does not do that for merely running a script, only when a file is imported.

(Previous versions used to store the cached bytecode as .pyc files that littered up the same directory as the .py files, but starting in Python 3 they were moved to a subdirectory to make things tidier.)

PYTHONDONTWRITEBYTECODE ---> If this is set to a non-empty string, Python won't try to write .pyc files on the import of source modules. This is equivalent to specifying the -B option.

Share Follow

answered Apr 21, 2020 at 10:12

Olive Hair
3,631 25 30

Is there any performance gain if the `file_name` is a file containing definitions of functions? – Anton Jun 30, 2022 at 16:03

from the official python tutorial Modules

7

> To speed up loading modules, Python caches the compiled version of each module in the `__pycache__` directory under the name `module.version.pyc`, where the version encodes the format of the compiled file; it generally contains the Python version number. For example, in CPython release 3.6 the compiled version of spam.py would be cached as `__pycache__/spam.cpython-36.pyc`.

from Python doc [Programming FAQs]

> When a module is imported for the first time (or when the source file has changed since the current compiled file was created) a .pyc file containing the compiled code should be created in a `__pycache__` subdirectory of the directory containing the `.py` file. The `.pyc` file will have a filename that starts with the same name as the `.py` file, and ends with `.pyc`, with a middle component that depends on the particular python binary that created it.

Share Follow

answered Mar 12, 2019 at 19:53

Yossarian42
1,930 17 14

6

Python Version 2.x will have **.pyc** when interpreter compiles the code.

Python Version 3.x will have **__pycache__** when interpreter compiles the code.

```
alok@alok:~$ ls
module.py  module.pyc  __pycache__  test.py
alok@alok:~$
```

Share Follow

answered Dec 7, 2019 at 3:43

Alok Tiwari
354 3 8

5

The python interpreter compiles the *.py script file and saves the results of the compilation to the `__pycache__` directory.

When the project is executed again, if the interpreter identifies that the *.py script has not been modified, it skips the compile step and runs the previously generated *.pyc file stored in the `__pycache__` folder.

When the project is complex, you can make the preparation time before the project is run shorter. If the program is too small, you can ignore that by using `python -B abc.py` with the `B` option.

Share Follow

answered Jun 21, 2018 at 7:05
Marcus Thornton
5,905 6 48 50

5

Execution of a python script would cause the byte code to be generated in memory and kept until the program is shutdown. In case a module is imported, for faster reusability, Python would create a cache .pyc (PYC is 'Python' 'Compiled') file where the byte code of the module being imported is cached. Idea is to speed up loading of python modules by avoiding re-compilation (compile once, run multiple times policy) when they are re-imported.

The name of the file is the same as the module name. The part after the initial dot indicates Python implementation that created the cache (could be CPython) followed by its version number.

Share Follow

answered May 28, 2019 at 10:13
TechFree
2,530 1 17 18

4

In 3.2 and later, Python saves .pyc compiled byte code files in a sub-directory named `__pycache__` located in the directory where your source files reside with filenames that identify the Python version that created them (e.g. script.cpython-33.pyc)

Share Follow

edited Dec 30, 2017 at 0:13
notpeter
1,034 10 16

answered Mar 30, 2017 at 15:37
Shashidhar Yalagi
51 1

How can I avoid the creation of "**pycache**" folder and all .pyc should be named same as .py file? – Ashwani Dec 16, 2019 at 8:23

0

does the byte code in `__pycache__` get automatically invoked the next time the application is started? I.E: if we run some application `main.py` for the first time and all the necessary modules get complied and stored in `pycache` then the next time are they automatically used even if I call `python main.py` ? or we will have to call `python _pycache_/main.pyc` ?

Share Follow

answered Apr 20, 2022 at 9:43
Vipin Menon
2,802 4 18 32

pycache is just a directory which contains .pyc file and it's just used for caching to make execution faster when we run next time like in browser which stores data using cookies so next time whenever we open

same page it doesn't have to make request for all css file again or think of it as Array containing data which uses similar concept of storing data using cache memory in OS. – Arya Singh Jun 14, 2022 at 17:59

Highly active question. Earn 10 reputation (not counting the association bonus) in order to answer this question. The reputation requirement helps protect this question from spam and non-answer activity.